

```
const express = require('express');
const cors = require('cors');
const fetch = (...args) => import('node-
fetch').then(({default: f}) => f(...args));

const app = express();
app.use(cors());
app.use(express.json());

const METAAPI_TOKEN =
process.env.METAAPI_TOKEN;
const METAAPI_ACCOUNT_ID =
process.env.METAAPI_ACCOUNT_ID;
const NEWS_API_KEY =
process.env.NEWS_API_KEY;

// ——— Helpers


---




---




---



function calcEMA(prices, period) {
  if (prices.length < period) return null;
  const k = 2 / (period + 1);
  let ema = prices.slice(0,
period).reduce((a, b) => a + b, 0) /
period;
  for (let i = period; i < prices.length;
i++) {
    ema = prices[i] * k + ema * (1 - k);
```

```

    }
    return parseFloat(ema.toFixed(5));
}

function calcRSI(closes, period = 14) {
    if (closes.length < period + 1) return
    null;
    let gains = 0, losses = 0;
    for (let i = 1; i <= period; i++) {
        const diff = closes[i] - closes[i - 1];
        if (diff >= 0) gains += diff; else
    losses -= diff;
    }
    let avgGain = gains / period;
    let avgLoss = losses / period;
    for (let i = period + 1; i <
closes.length; i++) {
        const diff = closes[i] - closes[i - 1];
        avgGain = (avgGain * (period - 1) +
Math.max(diff, 0)) / period;
        avgLoss = (avgLoss * (period - 1) +
Math.max(-diff, 0)) / period;
    }
    if (avgLoss === 0) return 100;
    const rs = avgGain / avgLoss;
    return parseFloat((100 - 100 / (1 +
rs)).toFixed(2));
}

function detectZones(candles) {
    const zones = [];
    const lookback = 5;
    for (let i = lookback; i < candles.length
- lookback; i++) {

```

```

    const slice = candles.slice(i -
lookback, i + lookback + 1);
    const highs = slice.map(c => c.high);
    const lows = slice.map(c => c.low);
    const maxHigh = Math.max(...highs);
    const minLow = Math.min(...lows);
    if (candles[i].high === maxHigh) {
        zones.push({ type: 'resistance',
price: candles[i].high, index: i });
    }
    if (candles[i].low === minLow) {
        zones.push({ type: 'support', price:
candles[i].low, index: i });
    }
}
// Merge nearby zones (within 2 USD for
gold)
const merged = [];
for (const z of zones) {
    const existing = merged.find(m =>
m.type === z.type && Math.abs(m.price -
z.price) < 2.0);
    if (existing) {
        existing.touches = (existing.touches
|| 1) + 1;
        existing.price = (existing.price +
z.price) / 2;
    } else {
        merged.push({ ...z, touches: 1 });
    }
}
return merged.filter(z => z.touches >=
1).sort((a, b) => b.touches -
a.touches).slice(0, 8);

```

```

}

function isLondonNYOverlap() {
  const now = new Date();
  const utcHour = now.getUTCHours();
  const utcMin = now.getUTCMinutes();
  const utcDecimal = utcHour + utcMin / 60;
  // London-NY overlap: 12:00-16:00 UTC
  (14:00-18:00 norsk sommertid)
  return utcDecimal >= 12 && utcDecimal <=
16;
}

function generateSignal(candles, zones,
ema50, ema200, rsi) {
  if (!candles.length || !ema50 || !ema200)
return null;
  const lastCandle = candles[candles.length
- 1];
  const price = lastCandle.close;
  const bias = ema50 > ema200 ? 'bullish' :
ema50 < ema200 ? 'bearish' : 'range';
  const overlap = isLondonNYOverlap();

  for (const zone of zones) {
    const zoneBuffer = 1.5; // $1.5 buffer
for gold
    const inZone = Math.abs(price -
zone.price) <= zoneBuffer;

    if (!inZone) continue;

    // Buy signal: price in support zone +
bullish bias + RSI not overbought

```

```

        if (zone.type === 'support' && bias ===
'bullish' && rsi < 65) {
            const sl = zone.price - zoneBuffer *
2;
            const tp = ema200 > price ? ema200 :
price + (price - sl) * 2;
            const rr = parseFloat(((tp - price) /
(price - sl)).toFixed(2));
            return {
                direction: 'buy',
                status: rr >= 1.5 ? 'confirmed' :
'armed',
                zone: zone.price,
                entry: price,
                stopLoss:
parseFloat(sl.toFixed(2)),
                takeProfit:
parseFloat(tp.toFixed(2)),
                riskReward: rr,
                bias,
                overlap,
                reasoning: `Pris i støttesone
($${zone.price.toFixed(2)}), bullish bias
(EMA50 > EMA200), RSI ${rsi} – ikke
overkjøpt`
            };
        }

        // Sell signal: price in resistance
zone + bearish bias + RSI not oversold
        if (zone.type === 'resistance' && bias
=== 'bearish' && rsi > 35) {
            const sl = zone.price + zoneBuffer *
2;

```

```

        const tp = ema200 < price ? ema200 :
price - (sl - price) * 2;
        const rr = parseFloat(((price - tp) /
(sl - price)).toFixed(2));
        return {
            direction: 'sell',
            status: rr >= 1.5 ? 'confirmed' :
'armed',
            zone: zone.price,
            entry: price,
            stopLoss:
parseFloat(sl.toFixed(2)),
            takeProfit:
parseFloat(tp.toFixed(2)),
            riskReward: rr,
            bias,
            overlap,
            reasoning: `Pris i motstandssone
($${zone.price.toFixed(2)}), bearish bias
(EMA50 < EMA200), RSI ${rsi} – ikke
oversolgt`
        };
    }
}
return null;
}

```

```
// — Routes
```

```

app.get('/health', (req, res) => res.json({
status: 'ok', time: new

```

```

Date().toISOString() }));

app.get('/api/market', async (req, res) =>
{
  try {
    if (!META-API_TOKEN ||
    !META-API_ACCOUNT_ID) {
      return res.status(500).json({ error:
'MetaApi credentials ikke satt. Sjekk
miljøvariabler.' });
    }

    const timeframe = 'H1';
    const limit = 150;
    const url = `https://mt-market-data-
client-api-
v1.london.agiliumtrade.ai/users/current/acc
ounts/${META-API_ACCOUNT_ID}/historical-
market-
data/symbols/XAUUSD/timeframes/${timeframe}
/candles?limit=${limit}`;

    const response = await fetch(url, {
      headers: {
        'auth-token': META-API_TOKEN,
        'Content-Type': 'application/json'
      }
    });

    if (!response.ok) {
      const text = await response.text();
      return
      res.status(response.status).json({ error:
'MetaApi feil: ${response.status}',

```

```

details: text });
    }

    const candles = await response.json();
    if (!candles || !candles.length) {
        return res.status(404).json({ error:
'Ingen candle-data mottatt fra MetaApi' });
    }

    const closes = candles.map(c =>
c.close);
    const ema50 = calcEMA(closes, 50);
    const ema200 = calcEMA(closes, 200);
    const rsi = calcRSI(closes, 14);
    const zones = detectZones(candles);
    const lastCandle =
candles[candles.length - 1];
    const signal = generateSignal(candles,
zones, ema50, ema200, rsi);
    const bias = ema50 && ema200 ? (ema50 >
ema200 ? 'bullish' : 'bearish') : 'range';

    res.json({
        symbol: 'XAUUSD',
        timeframe,
        price: lastCandle.close,
        open: lastCandle.open,
        high: lastCandle.high,
        low: lastCandle.low,
        ema50,
        ema200,
        rsi,
        bias,
        zones,

```

```

        signal,
        overlap: isLondonNYOverlap(),
        updatedAt: new Date().toISOString()
    });

    } catch (err) {
        console.error('Market error:', err);
        res.status(500).json({ error:
err.message });
    }
});

app.get('/api/news', async (req, res) => {
    try {
        if (!NEWS_API_KEY) {
            return res.json({ events: [],
warning: 'NEWS_API_KEY ikke satt' });
        }
        const today = new
Date().toISOString().split('T')[0];
        const tomorrow = new Date(Date.now() +
86400000).toISOString().split('T')[0];
        const url =
`https://finnhub.io/api/v1/calendar/economi
c?
from=${today}&to=${tomorrow}&token=${NEWS_A
PI_KEY}`;
        const response = await fetch(url);
        const data = await response.json();

        const relevant = (data.economicCalendar
|| []).filter(e =>
            ['USD', 'XAU',
'US'].includes(e.country) &&

```

```
    ['high', 'medium'].includes((e.impact
|| '').toLowerCase())
  ).map(e => ({
    time: e.time,
    event: e.event,
    country: e.country,
    impact: e.impact,
    minsUntil: Math.round((new
Date(e.time) - Date.now()) / 60000)
  }));

    const newsLocked = relevant.some(e =>
e.minsUntil >= 0 && e.minsUntil <= 60);
    res.json({ events: relevant, newsLocked
});
  } catch (err) {
    res.status(500).json({ error:
err.message });
  }
});

const PORT = process.env.PORT || 3001;
app.listen(PORT, () => console.log(`XAU
Signal Backend kjører på port ${PORT}`));
```